

*Multi – Level linked list*  
*–Insert After Element*  
*–Space Complexity and Algorithm*

**Program:**

```
Node *createNode(int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == nullptr)
    {
        cout << "Memory allocation failed." << endl;
        exit(1);
    }
    newNode->data = data;
    newNode->next = nullptr;
    newNode->child = nullptr;
    return newNode;
}

void insertAfterElement()
{
    int target, data, type;
    cout << "Enter the element to search for: ";
    cin >> target;

    // Simple search (Only searches main level for
    // simplicity in this demo)
    Node *temp = head;
    while (temp != nullptr && temp->data != target)
    {
        temp = temp->next;
    }
    if (temp == nullptr)
    {
        cout << "Element not found in main list." << endl;
        return;
    }
}
```

```

    cout << "Enter data to insert: ";
    cin >> data;
    Node *newNode = createNode(data);

    cout << "Insert as (1) Next Node or (2) Child Node? ";
    cin >> type;

    if (type == 1)
    {
        // Standard Linked List Insertion
        newNode->next = temp->next;
        temp->next = newNode;
        cout << "Inserted " << data << " as NEXT of " <<
target << endl;
    }
    else if (type == 2)
    {
        // Multilevel Insertion
        // If a child already exists, we link the new node
to the front of the child list
        newNode->next = temp->child;
        temp->child = newNode;
        cout << "Inserted " << data << " as CHILD of " <<
target << endl;
    }
    else
    {
        cout << "Invalid option." << endl;
        free(newNode);
    }
}
.....

```

```

int main()
{
    int choice;
    while (true)
    {

        cout << "4. Insert After Element (Link or Child)" <<
endl;
        cin >> choice;

        switch (choice)
        {
        case 4:
            insertAfterElement();
            break;
        default:
            cout << "Invalid choice" << endl;
        }
    }
    return 0;
}

```

## Space – Complexity

### Input – Space Complexity

#### **1. Parameter – only Input Space Complexity**

*This approach considers ONLY:*

- *function parameters*

*As there is no function parameter , but:*

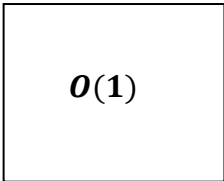
```
cin >> target;  
cin >> data;  
cin >> type;
```

*are considered part of the Parameter – only Input Space Complexity, because they are the direct external inputs supplied to the algorithm/function.*

*Even though they are not formal function parameters, they behave like input parameters.*

*These are still constant – sized scalar inputs.*

*So parameter – only input space remains constant:*



$O(1)$

## ***2. Holistic/Full Data Definition***

### ***Input Space Complexity***

*This approach considers:*

*Entire data structure processed by the algorithm.*

*Your function traverses the whole multilevel linked list.*

*Suppose total nodes in the structure =  $n$*

*Each node contains:*

```
int data;  
Node *next;  
Node *child;
```

*Each node size is constant.*

*So total memory required for the entire linked list:*

$$n \times O(1) = O(n)$$

*Therefore*

*Holistic / Full Data Definition Space Complexity:*

$O(n)$

### ***3. Auxiliary Space Complexity***

*Auxiliary space means:*

*Extra temporary memory used during execution excluding input storage.*

#### ***Variables Used***

*Integer variables*

```
int target, data, type;  
→ constant memory
```

*Pointer variables*

```
Node *temp ;  
Node *newNode ;  
→ constant memory
```

*Inside function of InsertAfterElement:*

```
int data;  
Node *newNode;
```

*Also in createNode(int data) function:*

```
Node *newNode = (Node *)malloc(sizeof(Node));
```

*Here ,both the newNode contains dynamically memory addresses, where as ,dynamically allocated memory :*

```
(Node *)malloc(sizeof(Node));
```

*itself is input space. But not:*

```
Node *newNode;
```

*Hence Node \* newNode;  $\rightarrow O(1)$  auxiliary space.*

*Hence ,Auxiliary Space is Constant  $\rightarrow O(1)$  i.e. constant space.*

---

# Total Space Complexity

Total Space Complexity = Auxiliary Space + Input Space

Based on the definition of input space you use, the total space complexity changes.

- **Common Interpretation (Parameter-only input):**

$$Space = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(1)}_{\substack{\downarrow \\ \text{Input}}} = O(1).$$

- **Holistic Interpretation (Full-data input):**

$$Space = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(n)}_{\substack{\downarrow \\ \text{Input}}} = O(n).$$

1) If Asked “Total Space Complexity” then natural answer is  
:  $O(n)$  [*Full Data + Auxiliary*]

2) *If not asked total space complexity then:*

In most academic and interview contexts, when asked for the space complexity of a function, the **auxiliary space complexity ( $O(1)$ )** is the expected answer.

In most academic DSA contexts and coding interviews, when someone simply asks:

“What is the space complexity?”

they usually mean:

|                            |
|----------------------------|
| Auxiliary Space Complexity |
|----------------------------|

unless they explicitly say:

- total space complexity
- overall memory usage
- data structure space
- input space
- storage complexity

So for your insertion function, the expected answer is usually:

|        |
|--------|
| $O(1)$ |
|--------|

because only constant extra working memory is used.

---

**Algorithm:**

***MultiINSAfterElem():***

1. *Declare three integer type variable target, data, type.*

2. *Write: ``Enter the element to search for``*

3. *Read target.*

4. *[ENTER NODE  $\geq$  1st Position?]*

*SET TEMPPTR := START*  $\left[ \begin{array}{l} \text{Set TemporaryPointer} \\ \text{to 1st Node which will} \\ \text{help to traverse the loop.} \end{array} \right]$

5.  $\left[ \begin{array}{l} \text{Traverse the List ,to point to ,the Node} \\ \text{containing the target if target was found.} \end{array} \right]$

*Repeat while TEMPPTR  $\neq$  NULL and INFO[TEMPPTR]  $\neq$  target :*

*SET TEMPPTR := LINK[TEMPPTR]*

*[End of Loop]*

6. *[If Out of Bound ?]*

*If TEMPPTR = NULL, THEN:*

*Write: ``Element not found in the main list``*

*RETURN*

*[End of If structure.]*

7. *Write: ``Enter data to insert:``*

8. *Read data.*

9. *SET NEWPTR := CALL CreateNode( data)*

---

## *Translating CreateNode(data)*

*((Node \*)malloc(sizeof(Node)) is availability of space for insertion of Data, hence, lets name it AVAIL.))*

### *CreateNode(data):*

#### *1. Allocate New Node*

*SET NewTempPTR := AVAIL.*

#### *2. Memory Allocated ?:*

*If NewTempPTR = NULL, then:*

*Write: " Error: Memory allocation failed."*

*Return and EXIT.*

*[End of If structure.]*

#### *3. Copy Data:*

*Set INFO[NewTempPTR] := data*

#### *4. Set Next section of Allocated Memory:*

*Set Link[NewTempPTR] := NULL*

#### *5. Set Child section of Allocated Memory:*

*Set childLink[NewTempPTR] := NULL*

#### *6. Return Stored Address in NewTempPTR*

*return NewTempPTR*

---

10. Write: ``Insert as (1) Next Node or (2) Child Node?``

11. Read type.

12. [ Insertion occurs in Main List]

if type = 1 , then :

**SET LINK[NEWPTR] := LINK[TEMPPTR]**

[ LINKING NEWNODE with the  
previous node after which  
newNode is to be placed  
in LINK LIST ]

**SET LINK[TEMPPTR] := NEWPTR**

[ LINKING NEWNODE with the  
NEXT NODE where this  
NEW NODE is to be placed  
in LINK LIST ]

Write : `` Inserted`` , data , ``as NEXT of`` , target .

[Insertion occurs in Child List]

Else if type = 2 , then :

**SET LINK[NEWPTR] := CHILD[TEMPPTR]**

[ LINKING NEWNODE with the  
child node of Node in the main list  
, where the target is present.  
[If any child node is linked with  
target node, then newNode will  
link to that child Node , that is  
linked with the target Node.] ]

***SET CHILD[TEMPPTR] := NEWPTR***

***The child section of the Target Node in the Main List will link to the New Node. This will make new child of the target node putting the new Node [New Child] before the older child of the target node.***

***Else:***

***Write: ``Invalid Option``***

***CALL FREE(NEWPTR)*** ***As, the NewNode is not attached to main list neither child, hence free the memory allocated by NewNode pointer.***

***[End of if structure]***

***13. END Of Algorithm***

---